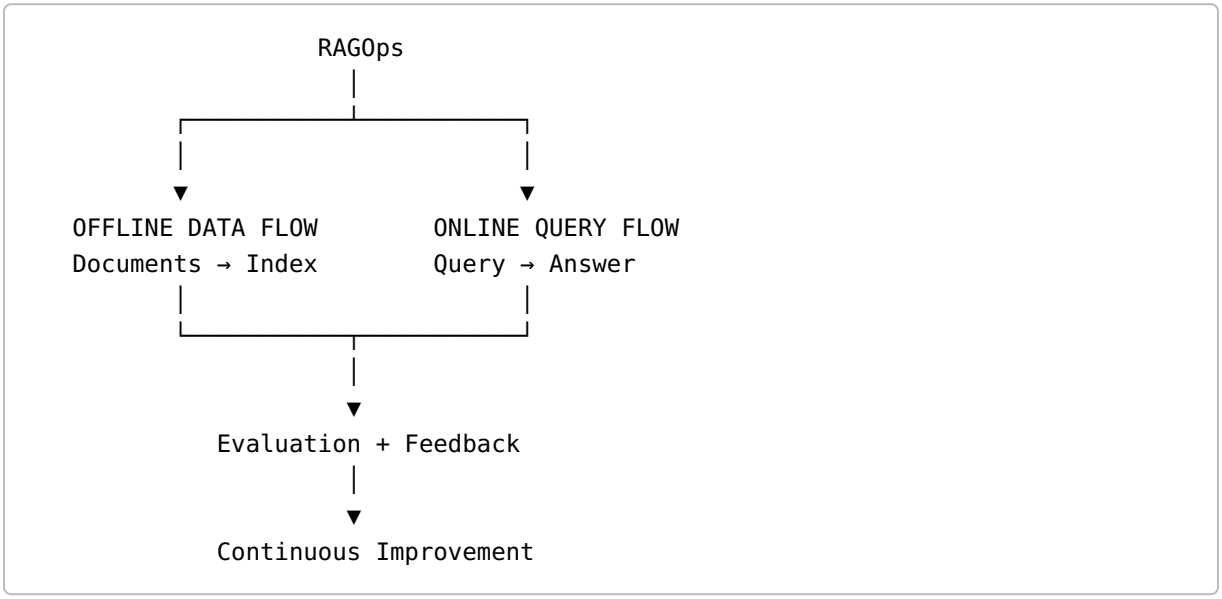
RAGOps — Data Flow Between Modules

1. Core Data-Flow Principle

The system has two major flows:



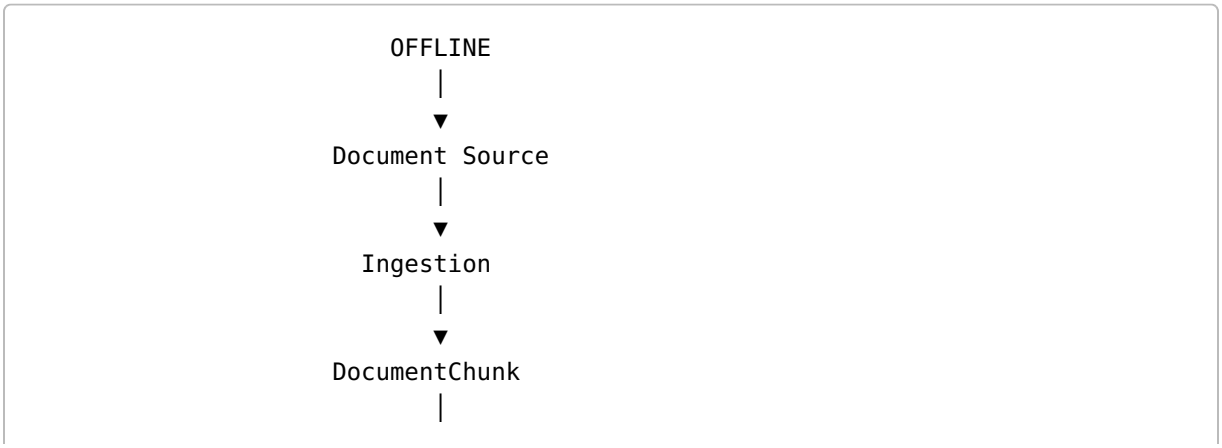
The most important architectural rule is:

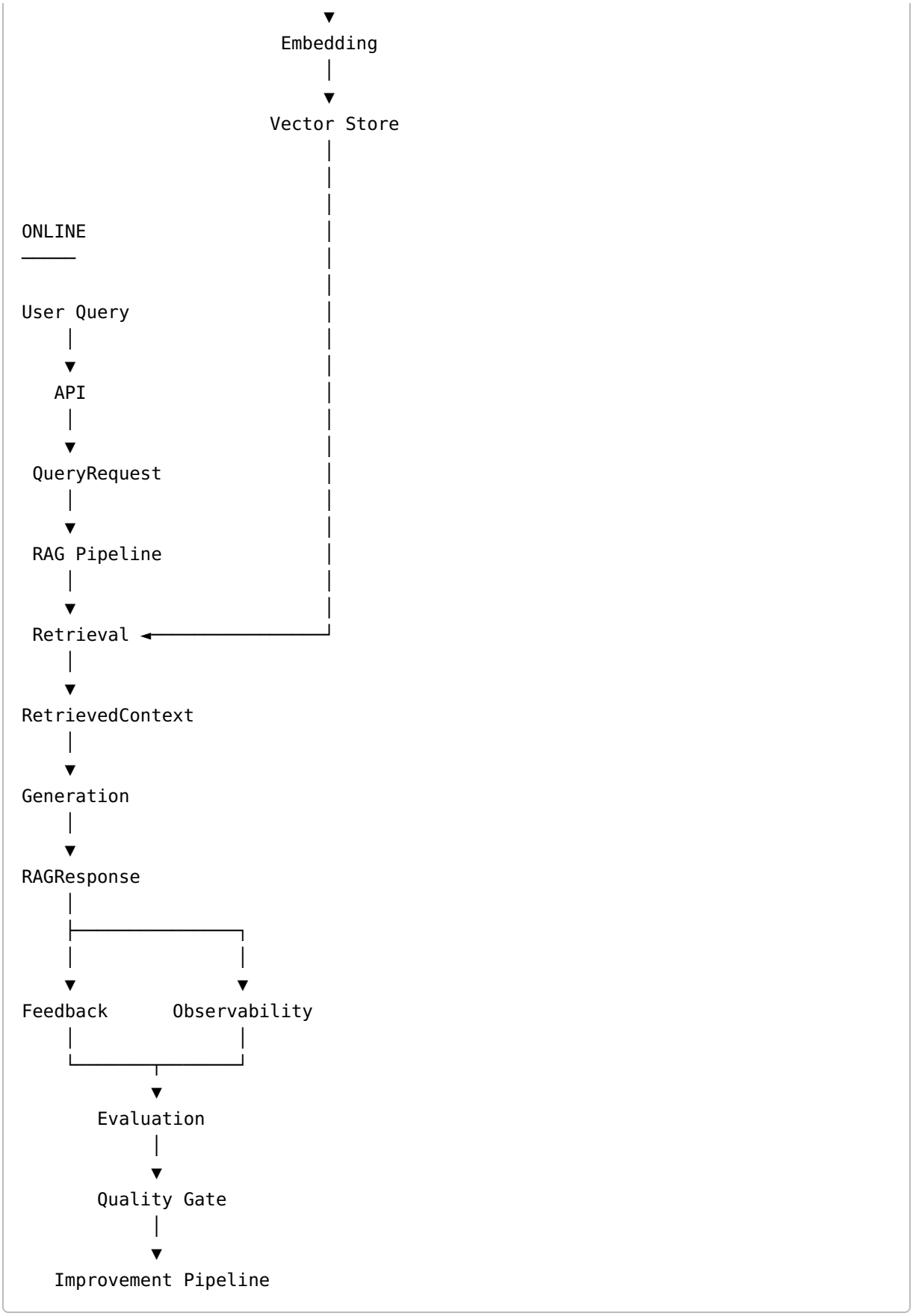
> **Modules exchange typed data objects, not implementation-specific objects.**

For example, `retrieval` should return a `RetrievedChunk`, not a raw Qdrant response.

---

2. End-to-End Data Flow




Embedding
Vector Store
ONLINE
User Query
API
QueryRequest
RAG Pipeline
Retrieval
RetrievedContext
Generation
RAGResponse
Feedback
Observability
Evaluation
Quality Gate
Improvement Pipeline

3. Shared Data Contracts

Create shared schemas in:

```
src/ragops/
└── core/
    └── types.py
```

Use Pydantic models or equivalent typed schemas.

These contracts become the "language" between modules.

3.1 `Document`

Created by the ingestion module.

```
Document(
    document_id="doc_102",
    source="engineering_handbook.pdf",
    version="v3",
    content="...",
    metadata={
        "author": "Engineering",
        "created_at": "2026-09-20"
    }
)
```

Flow:

```
Loader
  ↓
Document
```

4. Ingestion Data Flow

4.1 Loader → Cleaner

```
PDF / HTML / Markdown
        |
        ▼
     Loader
        |
        ▼
RawDocument
```

Example:

```
RawDocument(
    source="authentication.pdf",
    content="raw extracted text...",
    metadata={...}
)
```

The loader is responsible only for extracting content.

It does **not** clean or chunk the document.

4.2 Cleaner → Chunker

```
RawDocument
      |
      ▼
   Cleaner
      |
      ▼
CleanDocument
```

The cleaner handles:

- whitespace
- malformed text
- repeated headers
- irrelevant artifacts
- encoding issues

4.3 Chunker → Embedding

```
CleanDocument
      │
      ▼
    Chunker
      │
      ▼
DocumentChunk[]
```

Example:

```
DocumentChunk(
    chunk_id="auth_14_03",
    document_id="auth_14",
    text="OAuth 2.0 supports...",
    metadata={
        "page": 14,
        "section": "Authentication",
        "document_version": "v3"
    }
)
```

The chunk must preserve enough metadata to later answer:

> "Where did this answer come from?"

---

5. Embedding Data Flow

```
DocumentChunk[]
      │
      ▼
EmbeddingProvider
      │
      ▼
EmbeddedChunk[]
      │
      ▼
VectorStore
```

Conceptually:

```
EmbeddedChunk(
    chunk=DocumentChunk(...),
    embedding=[0.021, -0.183, ...],
    embedding_model="embedding-v2"
)
```

The embedding module owns:

```
text → vector
```

It should not know anything about:

```
LLM prompts
API requests
evaluation
```

---

6. Vector Store Data Flow

The vector-store boundary should look like:

```
EmbeddedChunk
      │
      ▼
VectorStore.upsert()
      │
      ▼
Vector Database
```

The vector store persists:

```
chunk_id
document_id
embedding
text
metadata
version
```

Later:

```
Query Embedding
      |
      ▼
VectorStore.search()
      |
      ▼
VectorSearchResult[]
```

Example:

```
VectorSearchResult(
    chunk_id="auth_14_03",
    score=0.87,
    metadata={...}
)
```

The vector-store implementation should hide whether you're using:

```
pgvector
Qdrant
FAISS
```

---

7. Online Query Data Flow

A user sends:

```
{
  "query": "How do I configure OAuth authentication?"
}
```

The API converts this into:

```
QueryRequest
```

```
QueryRequest(
    query="How do I configure OAuth authentication?",
```

```
    session_id="session_42"
)
```

Then:

```
API
 │
 ▼
QueryRequest
 │
 ▼
RAG Pipeline
```

---

8. Query Processing

The RAG pipeline passes the query to query transformation.

```
QueryRequest
      │
      ▼
QueryTransformer
      │
      ▼
ProcessedQuery
```

Example:

```
ProcessedQuery(
    original="How do I configure OAuth authentication?",
    search_query="OAuth authentication configuration",
    filters={}
)
```

Keep both:

```
original query
search query
```

because the original query is needed for generation and tracing.

---

9. Query → Embedding

```
ProcessedQuery
      |
      ▼
EmbeddingProvider
      |
      ▼
QueryEmbedding
```

Example:

```
QueryEmbedding(
    query="OAuth authentication configuration",
    vector=[...],
    model="embedding-v2"
)
```

---

10. Retrieval Flow

```
QueryEmbedding
      |
      ▼
VectorStore
      |
      ▼
VectorSearchResult[]
```

Example:

```
[
  auth_14_03  score=0.91
  auth_12_07  score=0.87
  auth_18_02  score=0.83
  auth_09_04  score=0.79
]
```

---

11. Retrieval → Reranking

The retrieval module then passes candidates to the reranker.

```
VectorSearchResult[]
        │
        ▼
     Reranker
        │
        ▼
RankedChunk[]
```

Example:

```
auth_14_03  → 0.96
auth_18_02  → 0.91
auth_12_07  → 0.82
```

The reranker should not generate an answer.

Its only job is:

> **Which retrieved chunks are most relevant?**

---

12. Retrieval → RAG Pipeline

The retrieval module returns a clean abstraction:

```
RetrievedContext(
    query="How do I configure OAuth authentication?",
    chunks=[
        RetrievedChunk(...),
        RetrievedChunk(...),
        RetrievedChunk(...)
    ]
)
```

Flow:

```
Query
 │
 ▼
Retrieval
 │
 ▼
RetrievedContext
 │
 ▼
RAG Pipeline
```

This is one of the most important module boundaries.

---

13. Context Builder

The RAG pipeline sends the retrieved context to:

```
context_builder
```

Flow:

```
Query
 +
RetrievedContext
      │
      ▼
ContextBuilder
      │
      ▼
PromptContext
```

Example:

```
PromptContext(
    query="How do I configure OAuth authentication?",
    context=[
        "...OAuth configuration...",
        "...client credentials...",
        "...redirect URI..."
```

```
    ]
)
```

The context builder controls:

- context ordering
- maximum context length
- deduplication
- metadata formatting
- citation references

---

14. Generation Flow

```
PromptContext
      |
      ▼
PromptBuilder
      |
      ▼
LLMClient
      |
      ▼
LLMResponse
```

Example:

```
LLMResponse(
    text="To configure OAuth...",
    model="model-v4",
    input_tokens=1250,
    output_tokens=210,
    latency_ms=932
)
```

The generation module should preserve metadata required for observability.

---

15. Generation → Response

The response builder combines:

```
LLMResponse
+
RetrievedContext
+
Request Metadata
```

into:

```
RAGResponse(
    answer="To configure OAuth...",
    sources=[
        Source(
            document="authentication-guide.pdf",
            page=14,
            chunk_id="auth_14_03"
        )
    ],
    metadata={
        "model": "model-v4",
        "retrieval_version": "v5"
    }
)
```

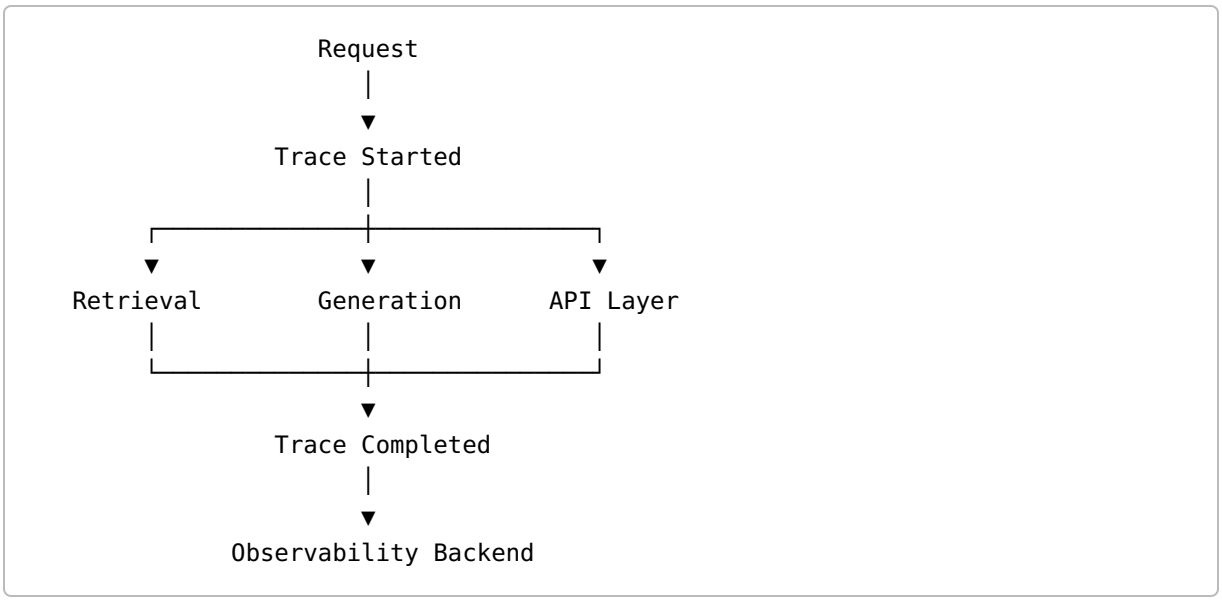
Then:

```
RAGResponse
     |
     ▼
   API
     |
     ▼
   JSON
     |
     ▼
   User
```

---

16. Observability Data Flow

Observability should happen **throughout the request**, not only after the response.



Each module emits telemetry.

Example:

```
TraceEvent(
    trace_id="trace_123",
    span="vector_search",
    duration_ms=82,
    metadata={
        "top_k": 10,
        "results": 10
    }
)
```

---

17. Evaluation Data Flow

Evaluation should reuse the **same RAG pipeline** used by production.

Do not create a separate "fake RAG implementation" for evaluation.

```
Golden Dataset
      |
      ▼
Evaluation Runner
      |
      ▼
```

```
RAG Pipeline
      │
      ├── Retrieval
      │
      └── Generation
      │
      ▼
Evaluation Outputs
      │
      ▼
Metrics Engine
      │
      ▼
EvaluationResult
```

Example:

```
EvaluationResult(
    dataset_version="golden-v4",
    rag_version="v12",
    retrieval_recall=0.91,
    faithfulness=0.94,
    answer_relevance=0.92,
    citation_accuracy=0.96
)
```

---

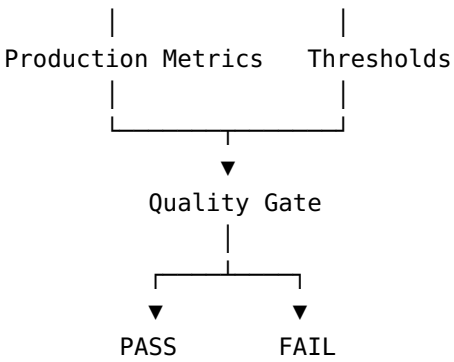
18. Evaluation → Quality Gate

The quality gate receives:

```
Candidate EvaluationResult
+
Production EvaluationResult
+
Threshold Configuration
```

Flow:

```
Candidate Metrics
       │
       ├──────────────┐
```



Output:

```
PromotionDecision(
    status="PASS",
    reasons=[
        "faithfulness improved",
        "retrieval recall above threshold",
        "latency within limit"
    ]
)
```

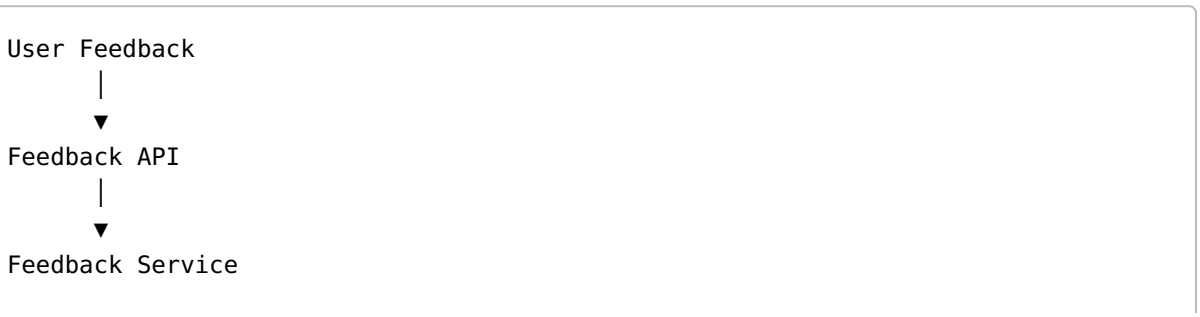
---

19. Feedback Data Flow

Production users can provide:

```
{
  "trace_id": "trace_123",
  "rating": "negative",
  "comment": "The answer ignored the latest documentation."
}
```

Flow:



```
      |
      ▼
Feedback Repository
      |
      ▼
PostgreSQL
```

Later:

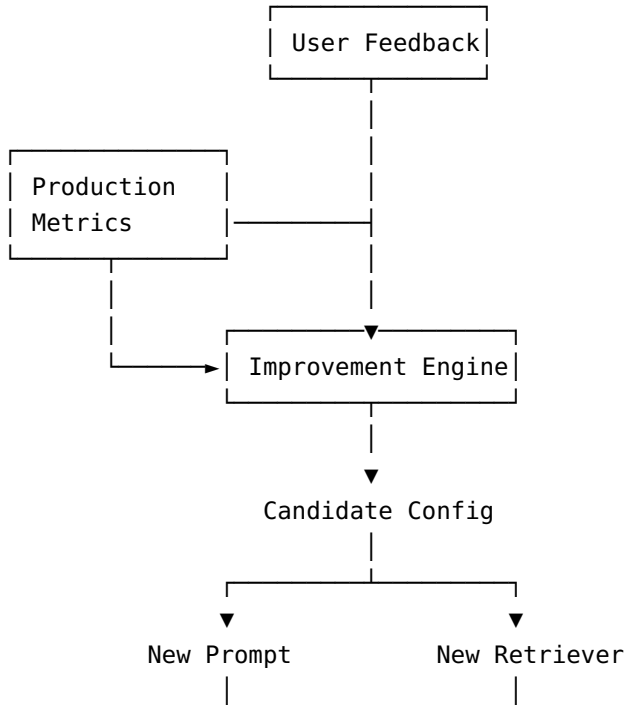
```
Feedback DB
     |
     ▼
Feedback Processor
     |
     ▼
Evaluation Dataset
```

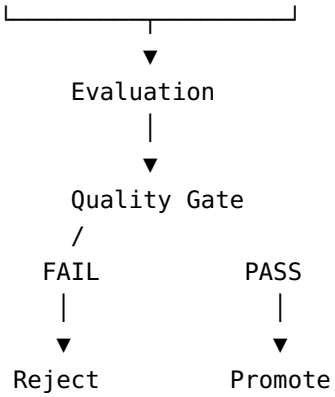
This is how production data eventually improves your evaluation set.

---

20. Continuous Improvement Flow

The improvement system combines multiple signals:





---

21. What Should NOT Flow Between Modules

Avoid these dependencies:

```
❌ API → Vector DB directly
❌ API → LLM directly
❌ Evaluation → Vector DB directly
❌ Evaluation → LLM directly
❌ Feedback → Prompt Builder directly
❌ Retrieval → FastAPI
❌ Generation → PostgreSQL
```

Instead:

```
API
 ↓
RAG
 ↓
Retrieval / Generation
 ↓
Infrastructure
```

And:

```
Evaluation
 ↓
RAG
```

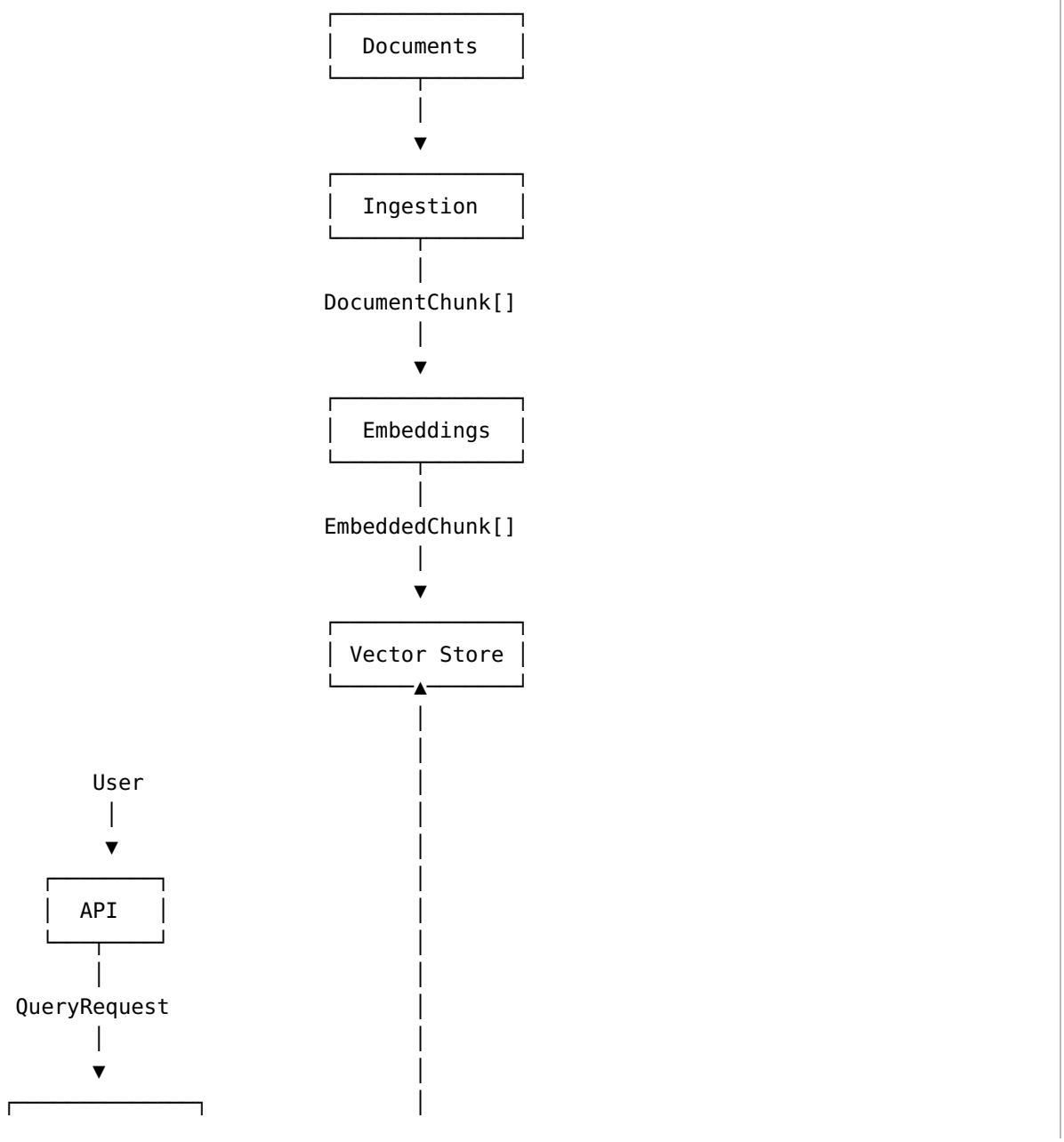
```
 ↓
Retrieval / Generation
```

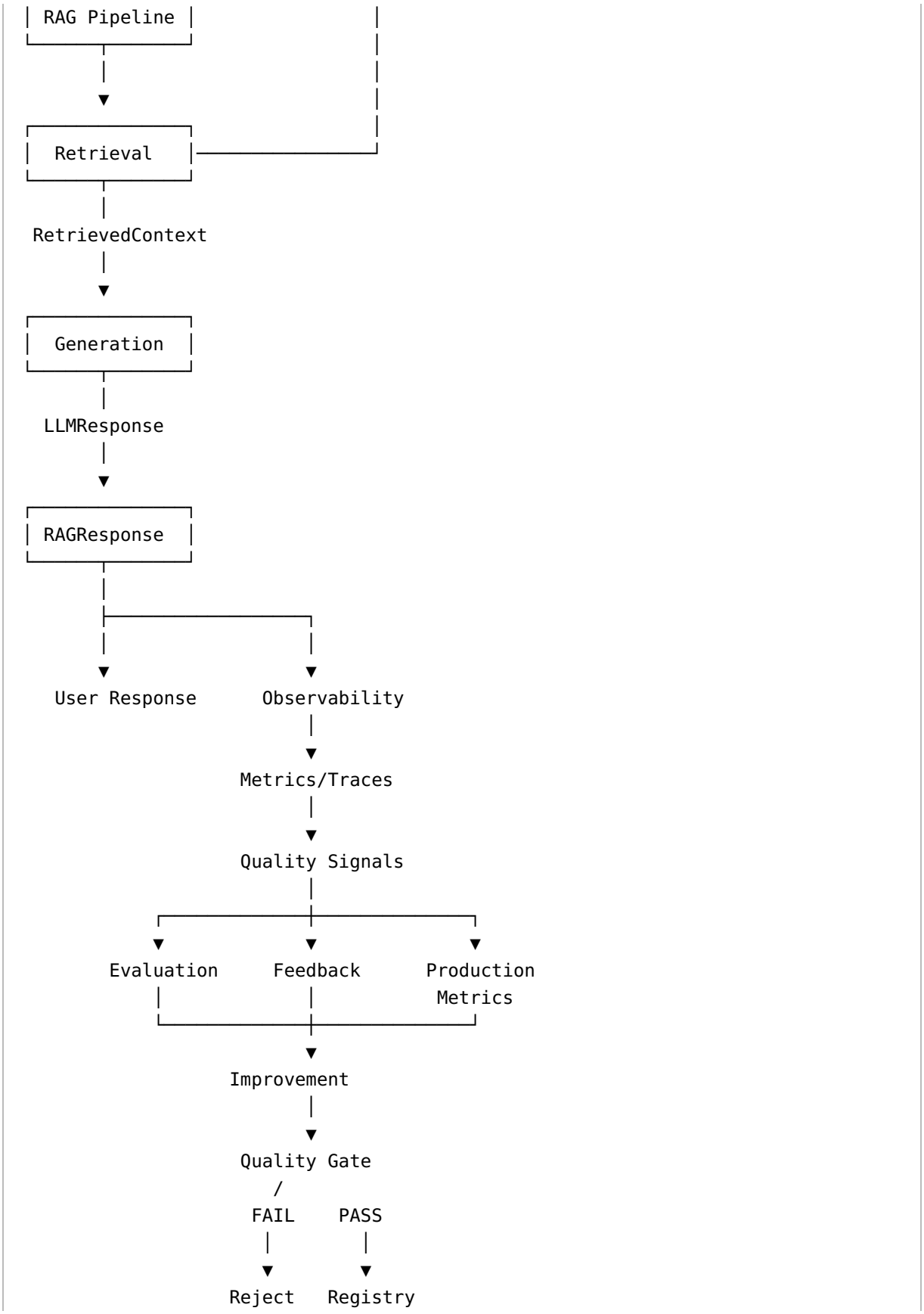
This keeps the architecture testable.

---

22. Complete Data-Flow Map

The final architecture can be summarized as:




RAG Pipeline
Retrieval
RetrievedContext
Generation
LLMResponse
RAGResponse
User Response
Observability
Metrics/Traces
Quality Signals
Evaluation
Feedback
Production
Metrics
Improvement
Quality Gate
FAIL
PASS
Reject
Registry

```
                    |
                    ▼
                 CI/CD
                    |
                    ▼
              Deployment
```

23. The Three Critical Contracts

If you implement nothing else initially, get these three boundaries right:

Contract 1 — Retrieval

```
Query
  ↓
Retriever
  ↓
RetrievedContext
```

Contract 2 — Generation

```
Query + RetrievedContext
          ↓
       Generator
          ↓
     RAGResponse
```

Contract 3 — Evaluation

```
Golden Dataset
      ↓
Same RAG Pipeline
      ↓
EvaluationResult
      ↓
QualityGate
      ↓
PromotionDecision
```

These three contracts give you a clean foundation for everything else.

24. Practical Rule for Implementation

When adding a new feature, ask:

> **"Which module owns this transformation?"**

For example:

```
PDF parsing       → ingestion
Chunking           → ingestion
Embeddings         → retrieval
Vector search      → retrieval
Reranking          → retrieval
Prompt construction→ generation/rag
LLM call           → generation
RAG orchestration  → rag
Evaluation         → evaluation
Tracing            → observability
User feedback      → feedback
Promotion decision → improvement
Database access    → storage
HTTP                → api
```

If a file starts doing two or three of these jobs, that's a signal that the module boundary is breaking down.